

```
GNU nano 8.7.1                                c.c
#include <stdio.h>
int main(){
int x=10;
printf("Value of x=%d\n",x);
printf("Virtual address of x=%p\n",(void*)&x);
return 0;
}
```

OUTPUT:

```
meghana@hp1:~$ gcc c.c -o c
meghana@hp1:~$ ./c
Value of x=10
Virtual address of x=0xffffce329dd4
meghana@hp1:~$ |
```

2)Virtual paging

```
GNU nano 8.7.1                                d.c
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {

    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address  : %p\n", (void *)main);
    printf("Data address   : %p\n", (void *)&global);
    printf("BSS address    : %p\n", (void *)&bss_variable);
    printf("Heap address    : %p\n", (void *)heap);
    printf("Stack address   : %p\n", (void *)&local);

    free(heap);

    return 0;
}
```

OUTPUT:

```

meghana@hp1:~$ nano d.c
meghana@hp1:~$ gcc d.c -o d
meghana@hp1:~$ ./d
Code address : 0xac449a6d08e8
Data address : 0xac449a6f0010
BSS address : 0xac449a6f0018
Heap address : 0xac449a6f1010
Stack address : 0xffffffff722138c
meghana@hp1:~$ |

```

3)Demand paging

```

GNU nano 8.7.1 e.c *
#include <stdio.h>

int main() {

    int page_table[4] = {1, -1, 3, -1};

    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}

```

OUTPUT:

```

meghana@hp1:~$ gcc e.c -o e
meghana@hp1:~$ ./e
Enter page number (0-3): 2
Page is already in memory.
Physical frame = 3
meghana@hp1:~$ |

```

4)Address translating

```
meghana@hp1: ~  
GNU nano 8.7.1 f.c  
#include <stdio.h>  
  
int main() {  
  
    int page_table[] = {3, 5, 1, 7};  
    int page_size = 100;  
  
    int virtual_address = 250;  
  
    int page = virtual_address / page_size;  
    int offset = virtual_address % page_size;  
  
    int frame = page_table[page];  
  
    int physical_address = frame * page_size + offset;  
  
    printf("Virtual Address = %d\n", virtual_address);  
    printf("Page Number      = %d\n", page);  
    printf("Offset           = %d\n", offset);  
    printf("Frame Number      = %d\n", frame);  
    printf("Physical Address = %d\n", physical_address);  
  
    return 0;  
}
```

OUTPUT:

```
meghana@hp1:~$ nano f.c  
meghana@hp1:~$ gcc f.c -o f  
meghana@hp1:~$ ./f  
Virtual Address = 250  
Page Number      = 2  
Offset           = 50  
Frame Number      = 1  
Physical Address = 150  
meghana@hp1:~$
```

5)dynamic memory

```
meghana@hp1: ~
GNU nano 8.7.1 dynamic.c
#include <stdio.h>
#include <stdlib.h>

int main() {

    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);

    return 0;
}
```

OUTPUT:

```
meghana@hp1:~$ gcc dynamic.c -o dynamic
meghana@hp1:~$ ./dynamic
10 20 30 40 50
```

6)Segmentation

```
meghana@hp1: ~
GNU nano 8.7.1
#include <stdio.h>

int main() {

    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}
```

OUTPUT:

```
meghana@hp1:~$ gcc null.c -o null
meghana@hp1:~$ ./null
before accessing memory
segmentation fault      ./null
meghana@hp1:~$
```